

Projeto MecaniQA

Autor: <Gustavo Prates Caetano>
<Matheus Costa Rodrigues>
<Nallanda Novato>
<Thainá Araujo Salgado>
<Vinicius Martins Lebrão>
<Vitor Hugo Lima Silva Bittencourt >

Vitória da Conquista, 2026

Agenda

O objetivo dessa apresentação é apresentar o desenvolvimento e validação da API RESTful criada para gerenciar peças e serviços mecânicos, cumprindo os requisitos da OAT 1.

1. Item:

Contextualização do problema.

2. Item:

Orientação a Objetos no Projeto.

3. Item:

Arquitetura e Padrão Singleton.

4. Item:

Controllers e API REST

5. Item:

Considerações Finais

Contextualização do problema



O Problema da Oficina:

Alta dificuldade na gestão manual de estoque de peças e no tabelamento de serviços prestados, gerando perda de tempo e erros de orçamento.

Objetivo da Fase 1:

Desenvolver o *Back-end* fundamental do sistema. Uma API RESTful capaz de centralizar o cadastro, listagem, atualização e exclusão (operações CRUD) de Peças e Serviços.

Infraestrutura Atual:

Nesta etapa, focamos na validação das regras de negócio. O armazenamento de dados opera puramente **em memória RAM** do servidor (Listas Java), dispensando banco de dados externo, como pedido na OAT.

Orientação a objetos no Projeto **unex**

- **Classes e Atributos**

- Definição das entidades Peça e Serviço

- **Modificadores de Acesso (Encapsulamento)**

- Atributos private para proteção dos dados.
- Métodos public (Getters e Setters).

- **Enums**

- Criação de CategoriaPeca para padronização
- Criação das constantes

```
1 package br.com.mecaniQA.api;
2
3 import java.time.LocalDateTime;
4
5 public class Peça { 16 usages  Gustavo Prates Caetano
6     private Long codigoUnico; 2 usages
7     private String nome; 2 usages
8     private String codigoBarras; 2 usages
9     private String fornecedorMarca; 2 usages
10    private int quantidadeEmEstoque; 2 usages
11    private Double precoCusto; 2 usages
12    private Double precoVenda; 2 usages
13    private LocalDateTime dataCadastro; 2 usages
14    private LocalDateTime dataUltimaAtualizacao; 2 usages
15    private CategoriaPeca categoriaPeca; 2 usages
16    private String tamanho; 2 usages
17    private String cor; 2 usages
18
19    public Long getCodigoUnico() { 1 usage  Gustavo Prates Caetano
20        return codigoUnico;
21    }
22
23    public void setCodigoUnico(Long codigoUnico) { 1 usage  Gustavo Prates Caetano
24        this.codigoUnico = codigoUnico;
25    }
26
27 package br.com.mecaniQA.api;
28
29 public enum CategoriaPeca { 3
30     MOTOR, no usages
31     SUSPENSAO, no usages
32     FREIOS, no usages
33     ELETRICA, no usages
34     ACESSORIOS no usages
35 }
```

Arquitetura e Padrão Singleton



- Características do **Singleton**
 - Construtor **private** (bloqueia o uso do "new")
 - Atributo **instance** como **private static**.
 - Método **getInstance()**, garantindo uma única instância global para acesso seguro aos dados
- Gerenciamento de estado em memória.
 - Uso de Listas (**List**) para armazenar os dados na porta 8081.

```
public class PecaRepository { 5 usages  Gustavo Prates Caetano

    private static PecaRepository instance; 3 usages

    private List<Peca> pecas; 5 usages

    private long contadorId; 2 usages

    private PecaRepository() { 1 usage  Gustavo Prates Caetano
        this.pecas = new ArrayList<>();
        this.contadorId = 1;
    }

    // O método getInstance() é a única forma de acessar o repositório.
    // Se a instância não existe, ele cria. Se já existe, ele devolve a mesma.
    public static PecaRepository getInstance() { 1 usage  Gustavo Prates Caetano
        if (instance == null) {
            instance = new PecaRepository();
        }
        return instance;
    }
}
```

Controllers e API Rest



- Os Controllers são responsáveis por receber e tratar as requisições HTTP realizadas pelos clientes através de **endpoints (rotas)** específicos.
- Na API REST, cada operação é associada a um **verbo HTTP**, como POST, GET, PUT e DELETE.
- No projeto, os Controllers fazem a comunicação entre as **requisições da Web** e a camada de **Repository**.
- Também são responsáveis por retornar as respostas adequadas, utilizando os **Status HTTP** definidos para cada operação.

```
@PostMapping @ Gustavo Prates Caetano
public ResponseEntity<Peca> cadastrar(@RequestBody Peca peca) {
    Peca pecaSalva = repository.salvar(peca);
    // Retorna 201 Created conforme exigido para criação com sucesso
    return ResponseEntity.status(HttpStatus.CREATED).body(pecaSalva);
}

@GetMapping @ Gustavo Prates Caetano
public ResponseEntity<List<Peca>> listarTodas() {
    // Retorna 200 OK
    return ResponseEntity.ok(repository.listarTodas());
}

// =====

@PutMapping("/{id}") @ Gustavo Prates Caetano
public ResponseEntity<Peca> atualizar(@PathVariable Long id, @RequestBody Peca pecaAtualizada)
{
    Peca pecaExistente = repository.buscarPorId(id);
}

@DeleteMapping("/{id}") @ Gustavo Prates Caetano
public ResponseEntity<Void> excluir(@PathVariable Long id) {
```

Rotas da API/Postman



Post: `http://localhost:8081/api/pecas`

Get: `http://localhost:8081/api/pecas`

Put: `http://localhost:8081/api/pecas/Indice`

Delete: `http://localhost:8081/api/pecas/Indice`

Considerações Finais



- **Arquitetura Implementada:** O Back-end foi construído aplicando fortemente os pilares de Orientação a Objetos.
- **Padrões de Projeto:** O uso do padrão Singleton permitiu a simulação eficaz de um banco de dados em memória.
- **Validação de Rotas:** A API RESTful demonstrou resiliência nos testes realizados via Postman, retornando os Status HTTP corretos.